

Real-time messaging platform, the scale follow-ups

front: six sentences first, then the follow-ups · Kyle's version of SD Q4

Anthropic question this is: SD Q4 chat (1point3acres, 11 discussions; Paul's opener is the 1:1 form), run here in the wide form with Kyle's four follow-ups: media, 100k groups, group read receipts, gateway crash. **Same family:** hellointerview chat and messaging (33 reports) · prachub "resilient chat" (its "ack that lies" is follow-up 4) · hack2hire Messenger, Slack-like · Tyler #5 (API-first). **Anthropic's own probes still to expect:** Redis Pub/Sub vs Kafka with the metadata argument, sharding by user vs conversation, the second device. **Kyle graded delivery:** six sentences first, no "is this okay?", ask for focus not the answer.

1 REQUIREMENTS 0–5 min · restate the problem as the north star

- **Six sentences, before anything else:** sockets to stateful chat servers behind an L4 balancer, registry = user → server = presence · durable log + inbox before anyone is told, then route or push · groups fan out through a batch service · media via presigned upload and CDN · read state is a monotonic watermark per member · retries safe by client_msg_id and ACK-after-write.
- **Ask:** group cap (~100; channels to 100k)? media? multi-device? presence accuracy? delete/edit? what is "delivered" with three devices?
- **A user can:** 1:1 and group · history synced across devices · see who is online · get offline messages on reconnect · delivered and read.
- **Technical challenges:** 500M DAU · 50B/day → 580k/s avg, ~2M/s peak · delivery <200 ms · no loss · availability over consistency (out-of-order accepted, render by ts) · fan-out cost · presence without writes.

2 CORE ENTITIES 5–9 min · the watermark replaces the matrix

- **Conversation**(id, type dm|group|channel, cap) · **Membership**(conv, user, role)
- **Message**(id monotonic per conv, conv_id, sender, client_msg_id, sender_ts, server_ts, body | media_ref) — immutable
- **Inbox**(recipient_device, message_id) until ACK — the durability backstop
- **ReadWatermark**(conv_id, user_id, highest_read_id) — monotonic, one row per member per conversation
- **Presence** = registry(user → server, TTL) · **Media**(id, uri, size, variants)

3 API 9–13 min · one endpoint per need

- **POST /conversations** · **POST|DELETE /conversations/{id}/members/{uid}**
- **POST /conversations/{id}/messages** {client_msg_id, body | media_ref, sender_ts} → {message_id, server_ts}, idempotent · **GET .../messages?since&limit**
- **POST /conversations/{id}/read** {highest_read_id} (server keeps the max) · **GET .../reads**
- **POST /media/uploads** {size, type} → presigned multipart URL · **GET /users/{id}/presence**
- **WS /ws** : message, receipt, presence frames; heartbeat 10–30 s

4 DATA FLOW 13–20 min · Alice to Bob, numbered

- Alice → L4 → her chat server → **append to Kafka + inbox row per recipient device** → **ACK to Alice only now** →
- registry: where is Bob? → Bob's server pushes over his socket; offline → push service, inbox waits →
- Bob's device ACKs → inbox row deleted, delivered receipt back → Bob opens the chat → his watermark advances, read state fans out.
- Persistence consumer: Kafka → Cassandra (wide rows by conversation); devices catch up with **since** on reconnect.

5 HIGH-LEVEL DESIGN 20–33 min · the board

- Clients → L4 → stateful chat servers (socket map) → registry (user → server, TTL) → Kafka → persistence consumer → Cassandra · inbox · push service · batch/fan-out service · media (presigned, transcoder, CDN) · watermark table · health and analytics.
- **The ACK rule:** acknowledge only after the durable write; an ACK from an in-memory broker is "the ack that lies".
- **Presence:** the registry entry with a TTL from the heartbeat; expiry is offline; never write last_seen per heartbeat.

6 DEEP DIVES 33–48 min · Kyle's four, then the card's probes

- **Media:** presigned multipart to object storage, chunked; transcode on the client where possible, else a transcoder makes variants; message carries a reference; recipients fetch via CDN; size limits. Chat path carries references, never bytes.
- **100k groups:** send → Kafka → batch/fan-out service splits members into batches, staggers, delivers per chat server (each fans out to its own users); pull on open for cold members; read receipts as counts.
- **Who read it:** one monotonic watermark per member per conversation; "read by N" = members with watermark ≥ N; opening the chat advances it, never back.
- **Gateway crashes after a send:** ACK sent → already durable, delivered from the inbox. No ACK → client keeps it pending and retries with the same client_msg_id; server dedupes. Idempotency.
- **Card's probes:** Redis Pub/Sub for hot fan-out (inbox is the backstop), Kafka for the durable log; shard by user for many small chats, by conversation for huge ones; second device = per-device inbox.
- **Bow:** the same six sentences you opened with.

The board at minute thirty

back: the answer with the four follow-ups placed on it

